

FloodGuard

AI-Powered Flood Intelligence Platform

Questions a reviewer, guide or examiner is likely to ask about this project — covering architecture, data ingestion, the event correlation engine, machine learning, retrieval-augmented generation, and the live realtime pipeline. Every answer is grounded in what the codebase actually does, including the parts that are weak.

The hardest questions are the ones about the model's accuracy and the mocked data source. Those are answered honestly rather than defensively — a reviewer will find them either way, and a candidate who raises them first is in a far stronger position.

Stack FastAPI · Next.js 15 · SQLAlchemy · scikit-learn · ChromaDB · Google Gemini

Sources OpenWeather (live) · IMD (mock) · NDMA/IMD/NDRF document corpus

Generated 29 August 2026 · from the working repository

Numbers Worth Memorising

If you are asked for a figure and hesitate, it undermines everything else you say. These are the ones that come up.

4,548 IndoFloods gauged flood events used for training	155 catchments — the GroupKFold grouping key	0.575 ROC-AUC, GroupKFold by gauge (unseen catchments)
0.611 ROC-AUC with catchment attributes (reference only)	0.496 ROC-AUC of the dataset we rejected — chance	2,677 RAG chunks indexed from 8 official PDFs
64.5 mm IMD heavy rain / 24h — the primary trigger threshold	115.6 / 204.5 very heavy / extremely heavy rain thresholds (mm)	6 hours event correlation window
60 s scheduler collection interval	0.42 s measured SSE push latency, cycle to browser	21 REST endpoints exposed by the gateway

LAYER	CHOICE	ONE-LINE JUSTIFICATION
Gateway	FastAPI	async ingestion, automatic OpenAPI, Pydantic validation
Storage	SQLite via SQLAlchemy 2.0	zero setup; models are portable to Postgres by changing one URL
Model	HistGradientBoosting	handles the NaNs in hydrology tables natively, no imputation step
Vector store	ChromaDB + ONNX MiniLM	runs locally, no embedding API key, no torch dependency
LLM	Gemini 2.5 Flash	free tier, fast, sufficient for grounded summarisation
Frontend	Next.js 15 App Router	file-based routing, server-side rewrites hide the API origin
Realtime	Server-Sent Events	one-directional traffic; survives proxies; auto-reconnects

1 · Product & Architecture

Opening questions. The examiner is establishing whether you understand the system as a whole before probing any one part.

1.1 What problem does FloodGuard actually solve? **CORE**

Flood information in India is scattered across IMD warnings, CWC river gauges, weather APIs and news feeds. Each source reports in isolation, so an official has to correlate them mentally. FloodGuard ingests those sources, detects which readings are worth reacting to, and merges related readings into a single **flood event** that evolves as evidence accumulates — with a confidence score, an ML severity estimate, and a plain-language summary. The output is one row per situation instead of dozens of disconnected observations.

1.2 Why model "events" rather than just showing the API responses? **CORE**

Because a flood is not an API response. Guwahati under heavy rain produces an IMD nowcast, a five-day district warning, a district rainfall figure and a basin forecast — four records describing one situation. Showing four rows makes the operator do the correlation. Modelling one event with four pieces of attached evidence puts that work in the system, and lets the event carry state the raw records cannot: a confidence score that rises as sources agree, a lifecycle, and an accumulating narrative.

1.3 Walk me through the data flow end to end.

Seven stages, all visible in the code:

- **Collect** — the scheduler fans out across IMD and OpenWeather endpoints every 60 s.
- **Adapt** — a per-source adapter normalises each response shape into one observation record. Raw JSON is retained.
- **Store** — observations are inserted immutably, with a content hash preventing duplicates.
- **Trigger** — each new observation is tested against source-specific rules; most produce nothing.
- **Correlate** — a fired trigger is matched to an open event in the same district within 6 h, or opens a new one.
- **Score** — the ML model estimates the probability of escalation to a severe flood.
- **Summarise & publish** — Gemini rewrites the event narrative, and an SSE message tells connected dashboards.

1.4 Why is the repository split into `apps/` and `services/` ?

Anything in `apps/` is a process you start — the API gateway, the web app, the mock source. Anything in `services/` is a pipeline that produces artifacts the gateway loads at runtime: the trained model file and the vector store. Previously `backend/`, `dashboard/`, `ml/` and `rag/` sat flat at the root with nothing signalling which were deployable. The split makes the deployment unit obvious.

1.5 Why does the frontend never call the weather APIs directly?

Three reasons. The API keys stay server-side and never enter the client bundle. There is no CORS negotiation, because the browser only ever calls same-origin `/api/*`, which Next.js rewrites to the gateway. And the gateway is where correlation and scoring happen — a browser calling IMD directly would have to re-implement the trigger logic, which is exactly the duplication that caused an earlier bug in this project.

Worth volunteering: the first version of the dashboard *did* re-implement IMD's rainfall thresholds in JavaScript. The same rule then existed in two places and could disagree. Removing it was one of the concrete improvements made during the rewrite.

1.6 What happens if one data source goes down?

Each endpoint fetch is individually guarded: a network error, non-2xx status or malformed JSON is logged and returns an empty list, so one dead endpoint cannot abort the collection cycle. The other sources still store observations and events still update. `GET /health` reports each subsystem's state, and the dashboard header shows whether the live stream is connected.

1.7 How would you add a completely new source, say CWC river gauges?

Write an adapter that emits the same observation dictionary, a collector that calls the endpoint, and a trigger rule for that source's semantics. Nothing downstream changes — the correlation engine, the scoring service and the API all operate on the normalised record and have no knowledge of where it came from. That was the explicit design goal of the adapter layer, and adding OpenWeather after IMD tested it: it required no change to correlation, storage, or the API.

2 · Ingestion & Data Modelling

Questions about the database and the discipline around raw data.

2.1 How do you stop the same reading being stored repeatedly? CORE

Every adapted record carries a `dedupe_hash` — a SHA-256 of the observation type, the location key, the observation *date*, and the meaningful values. It is a unique column. Polling the same unchanged reading every 60 s therefore inserts nothing. The date rather than the clock time is deliberate: an identical re-poll is not new information, but the same reading on a new day genuinely is.

The insert path also guards against duplicates *within* a single batch, not just against rows already in the database.

2.2 Why are observations immutable, and why keep the raw JSON?

Observations are the audit trail. If an event is later disputed, you must be able to show the exact payload that caused it — a mutated record cannot do that. Keeping raw JSON also means a future adapter improvement can re-derive better fields from data already collected, without having to re-fetch. Flood events, by contrast, are explicitly derived and mutable: they are conclusions, not evidence.

2.3 Explain your schema.

TABLE	HOLDS	MUTABLE?
<code>imd_observation</code>	every normalised reading + raw JSON + dedupe hash	no
<code>station</code>	station/district metadata, kept separate from readings	upserted
<code>tracked_location</code>	places users added beyond the built-in registry	yes
<code>flood_event</code>	derived situation: status, severity, confidence, risk, summary	yes
<code>event_evidence</code>	link table: which observation moved which event, and why	append-only
<code>prediction_result</code>	every scoring run, so trends can be charted	append-only

Separating metadata from time-series mirrors how CWC itself structures its API, which the architecture document notes was discovered by reverse-engineering their station endpoint.

2.4 Why is `event_evidence` a separate table rather than a JSON column on the event?

Because it carries its own attributes — the trigger reason, the contribution weight and the time it was attached — and it needs a uniqueness constraint so the same observation cannot be attached to the same event twice. It is also the thing an operator most wants to query: "why does the system believe this?" A JSON blob would make that answer unqueryable.

3 · Trigger Detection

The first place a reviewer will probe for arbitrary, invented numbers. The defence is that the numbers are not ours.

3.1 Where do your rainfall thresholds come from? **CORE**

They are IMD's own published daily rainfall classification, not values we chose:

CLASS	RANGE (MM / 24 H)	SEVERITY ASSIGNED
Heavy rain	64.5 – 115.5	moderate
Very heavy rain	115.6 – 204.4	high
Extremely heavy rain	≥ 204.5	severe

Similarly the colour codes are IMD's four-level scheme: 1 Green (no action), 2 Yellow (watch), 3 Orange (be prepared), 4 Red (take action). Anchoring on the source authority's own definitions means the thresholds are defensible rather than tuned to make a demo look good.

3.2 Why don't Green and Yellow warnings create events?

Because in IMD's scheme they are informational, not calls to action — Yellow means "watch", not "respond". Triggering on them would generate an event for almost every district almost every day, and an alerting system that fires constantly is one people stop reading. Orange is the first level at which IMD itself expects preparation, so that is where the system starts reacting.

3.3 OpenWeather uses a different rainfall rule from IMD. Why the inconsistency? **HARD**

It is not inconsistency, it is a units mismatch that would be a bug if ignored. IMD's 64.5 mm figure is a **24-hour total**. OpenWeather reports rain over the last **1 or 3 hours**. Applying the daily threshold to a 3-hour reading would mean almost nothing ever triggers, even during an intense downpour. So observed OpenWeather rain triggers at a quarter of the daily figure — a rate that, sustained, clearly exceeds it.

The OpenWeather *forecast* rule is different again: we sum the next eight 3-hourly slots to get a genuine 24-hour accumulation, and that *is* compared against the full IMD thresholds, because it is now the same quantity.

3.4 Do all triggers count equally?

No. Each trigger carries a contribution weight reflecting how much it should move belief: a Red nowcast contributes 0.30, an observed heavy-rain reading 0.25–0.35, a basin forecast 0.15, and atmospheric conditions (low pressure with saturated air) only 0.10 — the last is supporting evidence and is never enough to open an event on its own.

4 · Event Correlation Engine

The architecture document calls this "the heart" of the platform. Expect the most detailed questioning here.

4.1 How do you decide two observations describe the same flood? CORE

Spatial and temporal matching. A fired trigger looks for an event in the **same district** that is still open and whose `last_seen` falls within the **6-hour correlation window**. If one exists, the observation attaches to it as evidence; otherwise a new event opens. District-level matching is appropriate because that is the granularity at which IMD issues warnings and at which response is actually coordinated.

4.2 Why six hours?

It has to be longer than the gap between related signals but shorter than the gap between genuinely separate events. Sources report on different cadences — a nowcast covers 3 hours, district rainfall is daily, a basin forecast is issued once a day — so a window shorter than a few hours would fragment one situation across several events. Six hours covers a monsoon burst and its associated warnings while still letting a fresh storm two days later open its own event. It is configurable via `CORRELATION_WINDOW_HOURS`, which matters because the right value is an operational judgement, not a constant of nature.

4.3 What is the confidence score and how does it grow? CORE

It represents **how much independent evidence agrees** that this event is real — deliberately distinct from severity, which is how bad it is. A new event starts at its trigger's contribution plus 0.10. Each additional piece of evidence adds its own contribution, capped at 0.98.

Concretely, an event that has an IMD Red nowcast, a Red five-day warning and an 85 mm district rainfall reading is far more credible than one resting on a single forecast — and the score reflects that, which is what drives the status ladder.

4.4 Why is there a cap at 0.98 rather than 1.0?

Because the system is inferring from remote sensor readings, not observing a flood. Reserving the last two points is an honest admission that no amount of corroborating telemetry constitutes certainty about conditions on the ground. It also stops the UI ever showing "100% confident", which would invite more trust than the evidence supports.

4.5 Explain the diminishing-returns rule on evidence. HARD

This fixed a real bug. IMD's basin QPF response contains one row per **sub-basin** — the Brahmaputra returns Lohit, Dibang, Subansiri and Kopili. All four are the same forecast, split geographically. The first implementation attached all four at full weight, pushing an event from 0.25 to 0.85 confidence on what was really **one signal counted four times**.

Now each additional piece of evidence *of the same observation type* contributes at half the previous one: 0.15, then 0.075, 0.0375, 0.019. Corroboration from a genuinely different source still counts fully. After the fix, that same event settled at 0.53 — "developing" rather than "high risk", which is the correct reading of a lone basin forecast.

Follow-up to expect: "how did you find that?" — the Dibrugarh event was showing 0.85 confidence with four pieces of evidence that were all QPF rows. Inspecting the evidence trail made the double-counting obvious, which is itself an argument for storing the trail.

4.6 Describe the event lifecycle.

`potential` → `developing` → `high_risk` → `active` → `resolved`. Status is derived from confidence and severity together, not confidence alone: reaching `active` requires confidence ≥ 0.75 *and* severity of high or severe, so a well-corroborated but mild situation does not masquerade as an emergency. Events with no new evidence for 24 hours are automatically resolved.

4.7 How is the risk score computed, and why is it not just the model output? HARD

Risk is a 0–100 blend. Before the model has scored an event it is 55% confidence and 45% observed severity. Once scored, the weights become 40% confidence, 35% severity and **25% model probability**.

The model is deliberately a minority contributor. Its cross-validated ROC-AUC is 0.575 — better than chance but weak — so letting it dominate a number that officials might act on would misrepresent how much it actually knows. The rule-based component, built on IMD's published thresholds, is the more defensible signal and carries the majority weight.

4.8 What are the known limitations of this correlation approach?

- Matching is by district name, so a flood spanning a district boundary produces two events rather than one. Genuine spatial correlation would use the coordinates already stored on each observation.
- There is no merge operation — two events that should become one cannot currently be joined.
- Contribution weights are hand-set. With labelled outcomes they could be fitted rather than assumed.
- The 24-hour resolution rule is time-based, not evidence-based; a receding flood is not detected as receding.

5 · Machine Learning

The most important section to get right. There is a weak result here, and the strongest position is to present it first, with the reasoning that led to it.

5.1 What does your model predict, and on what data? CORE

Given the rainfall that has already fallen, it estimates the probability that a flood event escalates to a **severe** flood. Training data is **IndoFloods**: 4,548 real gauged flood events across 155 Indian catchments, each labelled **Flood** or **Severe Flood** (35.8% positive), joined to observed antecedent rainfall accumulations over 1–10 days.

The model is a `HistGradientBoostingClassifier`, chosen because these hydrology tables contain genuine missing values and it handles NaNs natively rather than requiring an imputation step that would invent data.

5.2 You had a dataset called `flood_risk_india.csv` with 10,000 labelled rows. Why didn't you use it?

HARD

Because its labels are not related to its features. Trained and evaluated properly, it reaches **ROC-AUC 0.496** — indistinguishable from a coin flip — with 49.7% accuracy on a 50.6% positive class.

The decisive evidence was the feature importances: the seven main numeric features each landed at almost exactly 0.12, near-perfectly uniform. A real signal produces uneven importances, because some variables matter more than others. Uniform importance across unrelated variables is the signature of a model splitting on noise, which is what a synthetic dataset with randomly assigned labels produces.

The training script is **still in the repository**, along with its metrics file. It is kept deliberately as the evidence for the rejection — deleting it would leave the decision undocumented.

This is a strength, not an admission. Shipping a 0.496 model labelled "flood risk prediction" would have looked more impressive on a slide and been actively dangerous. Detecting that a dataset is unusable is a real skill, and the alternative model was found and trained instead.

5.3 Why GroupKFold instead of ordinary cross-validation? CORE

Because the 4,548 events come from only 155 catchments — roughly 29 events per catchment. Under random splitting, events from the same gauge appear in both train and test folds, so the model can memorise a catchment's typical behaviour and score well without generalising. That is optimistic leakage.

Grouping by `GaugeID` guarantees every catchment in the test fold is one the model has never seen — which is exactly the situation it faces when scoring a district live. The reported 0.575 is therefore the honest number.

5.4 Your in-sample ROC-AUC is 0.967 but cross-validated it is 0.575. Isn't that massive overfitting? HARD

Yes, and that gap is precisely why the cross-validated figure is the one reported everywhere — in the metrics file, in `/predictions/model`, and in the API response. A gradient-boosted ensemble with 300 iterations will fit training data very closely; the in-sample number is recorded only to make the contrast visible, and is explicitly labelled "optimistic, for reference only" in the training output.

Reducing the gap would mean stronger regularisation, fewer iterations, or early stopping on a grouped validation fold. That is a legitimate next step, though on a signal this weak it would likely trade in-sample fit for very little out-of-sample gain.

5.5 ROC-AUC 0.575 is barely above chance. Why ship it at all? **HARD**

Three reasons, and the honesty of the framing matters more than the number.

- It is **consistently** above chance across five grouped folds on real gauged data, unlike the rejected dataset which sat at chance. A weak real signal is a different thing from no signal.
- It is weighted at only **25%** of the risk score. The headline number is driven by IMD's published thresholds.
- Its limitation is stated in the model metrics, returned in every prediction response, and surfaced through the API — a reviewer does not have to discover it.

The alternative — omitting ML entirely — would have been defensible. Presenting 0.575 as if it were 0.95 would not.

5.6 What features does the served model use, and why those?

Ten observed rainfall accumulations (T1d ... T10d), a month term for seasonality, and three engineered ratios:

- `burst_1_3` — day-1 rain as a fraction of 3-day rain
- `burst_3_10` — 3-day as a fraction of 10-day
- `late_accum` — 10-day total minus 3-day total

The burst ratios exist because the same 10-day total behaves very differently depending on its shape. 200 mm falling in one afternoon onto dry ground produces flash flooding; the same 200 mm spread over ten days largely infiltrates. A model given only totals cannot distinguish those two cases.

5.7 You had catchment attributes that raised AUC to 0.611. Why aren't you using them? **HARD**

Because they cannot be resolved at inference time. Catchment characteristics — drainage area, relief ratio, stream order — are keyed to IndoFloods gauge IDs. For an arbitrary district there is no mapping from that district to a gauge, so serving that variant would require fabricating catchment attributes for live locations.

A model that scores 0.611 on inputs you cannot supply is worth less than one that scores 0.575 on inputs you can. Both variants are trained and both numbers are recorded; the served one is the honest choice. Unlocking 0.611 requires the CWC station registry, which is on the backlog.

5.8 Did you check for target leakage?

Yes, and it changed the feature set. `Peak Flood Level` and `Peak Discharge` are available in the events table and would raise the score substantially — but "Severe Flood" is almost certainly labelled *from* peak level. Including them would be predicting the label from a proxy for itself.

Both are excluded. The model sees only rainfall that fell **before** the outcome, which is also the only information available when forecasting. Latitude and longitude were excluded from the earlier model for a related reason — raw coordinates invite memorising locations rather than learning hydrology.

5.9 Live data doesn't give you T1d...T10d. How do you bridge that? **HARD**

By anchoring and interpolating, and then saying so in the response. IMD supplies a daily total, a weekly total and a cumulative total. Those become the day-1, day-7 and day-10 anchors; the intermediate days are linearly interpolated, and accumulations are kept monotonically non-decreasing.

This is an approximation, so `/predictions` returns a `feature_provenance` map marking every feature as **observed**, **interpolated** or **imputed**. If no live rainfall exists for a district, the training medians are used and labelled as imputed. The consumer can therefore see how much of any given score rests on real measurement.

5.10 How would you actually improve this model?

- **Wire CWC.** River level and discharge are the physically causal variables for flooding; rainfall alone is an upstream proxy. This is the single largest available gain.
- **Resolve catchments** for live districts, unlocking the 0.611 feature set.
- **Add antecedent soil moisture** — the same rainfall floods saturated ground and not dry ground, and this is the main physical driver the current features cannot see.
- **Regularise harder** to close the 0.967 / 0.575 gap.
- **Calibrate probabilities** (Platt or isotonic) so a "60%" means what it says, which matters if the number ever drives a decision threshold.

5.11 Why not deep learning?

4,548 rows with 14 features is firmly in gradient-boosting territory. A neural network would have more parameters than the dataset can constrain, and would almost certainly do worse while being harder to explain. On tabular data of this size, boosted trees are the stronger baseline — and given the signal is weak, the priority is better *features*, not a larger model class.

6 · Retrieval-Augmented Generation

Expect questions on why RAG rather than fine-tuning, and on how hallucination is prevented.

6.1 Describe your RAG pipeline end to end. CORE

Eight official PDFs from NDMA, IMD and NDRF are parsed page by page with PyMuPDF. Text is cleaned and chunked paragraph-aware at roughly 1,200 characters with 200 characters of overlap, and each chunk keeps its source file, document title and page number. Chunks are embedded with ChromaDB's bundled ONNX MiniLM and stored with cosine distance — **2,677 chunks** in total.

At query time the system runs hybrid retrieval, filters for duplicates and document diversity, and passes the surviving passages to Gemini with a system prompt requiring it to answer only from that context and cite each claim as [Title, p.N] .

6.2 Why RAG rather than fine-tuning a model on these documents? CORE

Because the requirement is **citation**, not fluency. An official acting on evacuation guidance needs to know which document and page it came from, so it can be verified — a fine-tuned model absorbs the text into weights and can produce plausible guidance with no traceable source. RAG also updates by re-indexing when NDMA publishes a revision, whereas fine-tuning would require retraining. And on a corpus of eight documents there is nowhere near enough text to fine-tune usefully.

6.3 Why did you write a second ingestion script instead of using the existing one? HARD

The original `ingest.py` is the higher-quality path: `unstructured` in hi-res mode with Tesseract OCR, table extraction to HTML, and a vision LLM captioning every chart and map. It also needs system binaries installed and takes 30–45 minutes.

`fast_ingest.py` pulls the embedded text layer with PyMuPDF and embeds locally — about two minutes, no OCR, no API key, no torch. The tradeoff is stated in the file's docstring: **text inside scanned pages and charts is not captured**. All eight documents in this corpus have usable text layers, so nothing was lost here, but for a scanned document the OCR path remains the correct tool. Both are kept.

6.4 Why 1,200-character chunks with 200 overlap?

Large enough to hold a complete procedural clause — evacuation steps lose meaning split across chunks — and small enough that five of them fit comfortably in a prompt without diluting the relevant passage. The overlap prevents a rule being cut exactly at a boundary. Chunking is paragraph-aware first and only falls back to a hard sliding window for a single oversized paragraph, so clause boundaries are respected where they exist.

6.5 You use hybrid retrieval. What was wrong with pure vector search? HARD

A concrete failure exposed it. Asked *"What does an Orange IMD warning mean?"*, pure vector search returned four near-identical chunks — the same page footer (*"Red color warning means Take Action"*) reprinted on pages 5, 20, 24 and 25 of one bulletin. Gemini correctly answered that the context did not contain the answer.

But the answer **was** in the corpus: IMD SOP page 29, Table 1.11, defines the colour criteria explicitly. MiniLM simply ranked a topically-similar disclaimer above a criteria table whose surrounding language looks nothing like the question.

The fix has three parts:

- **Keyword pass** — salient query terms are matched literally against chunk text and merged with the vector hits, each matched term adding 0.12 to that passage's score.
- **Near-duplicate filtering** — a normalised 160-character fingerprint drops boilerplate reprinted across pages.
- **Per-document cap** — at most two passages from any one PDF, so a single document cannot fill the context.

After the change the same question returns SOP pp. 29 and 263 and yields a correct, cited answer.

6.6 How do you prevent the model hallucinating? CORE

Four layers. The system prompt instructs the model to answer **only** from the supplied context and to say plainly when the answer is absent. It must cite [Title, p.N] after each claim, which makes an invented statement visibly unsupported. It is explicitly told never to invent phone numbers, thresholds or procedures — the specific things most dangerous to fabricate in a disaster context. And temperature is 0.2 rather than a creative setting.

The behaviour is observable: before the retrieval fix, the model *did* say the context did not contain the answer rather than inventing one. That refusal was evidence the grounding works.

6.7 What happens if there is no Gemini key, or the call fails?

Retrieval still runs and the top passages are returned verbatim, labelled as unsummarised source text rather than presented as an answer. Retrieval is the part that must always work; generation is a convenience layer over it.

Importantly, a **failing** key is treated differently from **no** key. The client surfaces Google's actual error message and tags a reason — `bad_key`, `bad_model`, `rate_limited`, `blocked`, `truncated` — and the chat interface shows a red banner. An earlier version reported every failure identically, so a mistyped key looked like an intentional no-key configuration.

6.8 How would you evaluate retrieval quality properly? HARD

Currently it is evaluated by spot-checking, which is a genuine weakness. A proper evaluation would need a labelled set of maybe 50 questions with known correct source pages, then measuring **recall@k** (does the right page appear in the top k) and **MRR**. Answer quality would need faithfulness scoring — checking each generated claim is supported by a cited passage — which frameworks like RAGAS automate.

Without that harness, the hybrid-retrieval change is justified by a single reproducible failure case rather than by an aggregate metric. That is honest but not sufficient for production.

6.9 Why ChromaDB and a local embedding model?

Chroma is embedded — no server to run — and persists to a directory, which suits a store rebuilt from source PDFs. Its bundled ONNX MiniLM is the deciding factor: embeddings run locally, so there is **no embedding API key and no per-query cost**, and the 80 MB ONNX runtime avoids pulling in PyTorch. For 2,677 chunks this is comfortably fast. At millions of chunks the answer would change to pgvector or a dedicated vector database, which also matches the architecture document's stated Postgres direction.

6.10 Why do greetings get a different response path?

Because running semantic search on "Hi" returns whichever passage happens to be least dissimilar to it, which reads as broken. Greetings and pleasantries are detected and answered with a short capability statement instead. This also fixed a validation bug: the endpoint originally required a three-character minimum, so "Hi" returned HTTP 422 — rejecting a greeting with a validation error is a worse answer than replying to it.

7 · LLM Integration

7.1 Why a raw REST client rather than the Google SDK?

The surface actually needed is a single `generateContent` POST. A thin `httpx` wrapper is one dependency fewer, makes the retry and error-classification logic explicit rather than hidden inside a library, and keeps the failure modes visible — which mattered, because carrying the real error message to the user was a specific requirement.

7.2 What is `thinkingBudget: 0` doing?

Gemini 2.5 models perform extended reasoning before producing output, and that reasoning consumes the output token allowance. For short grounded answers over retrieved passages the budget can be spent before any text is emitted, yielding a 200 response with empty content. Setting it to zero disables that. Gemini 2.5 Pro rejects a zero budget, so a 400 mentioning "thinking" triggers one retry with the field removed.

7.3 How do you handle rate limits and transient failures?

429 and 5xx responses are retried up to three times with exponential backoff, since free-tier rate limits are common. Authentication and model-name errors are **not** retried — they will never succeed, and retrying only delays a clear error message. A 200 with no text is classified separately into safety-block, truncation or empty, because those need different fixes.

7.4 How does the AI event summary avoid inventing facts?

It is generated from the event's own **evidence trail** — the list of attached observations with their trigger reasons and weights — plus the computed confidence, severity and risk. The prompt restricts it to that evidence and asks it to state confidence honestly, including when the evidence is thin. Because the input is the same record shown in the UI, a reader can check the summary against what it was built from.

8 · Backend, Frontend & Realtime

8.1 Why SQLite? Isn't that a toy database? **HARD**

It is the right choice for the current write volume — one collection cycle a minute — and it needs no setup, which matters for a project that has to be runnable by someone else in minutes. The important point is that nothing is *coupled* to it: every model is plain SQLAlchemy 2.0 and the migration to PostgreSQL is a `DATABASE_URL` change, which the config file documents. The architecture document specifies Postgres with pgvector for production, and the schema was written to move.

8.2 Why Server-Sent Events instead of WebSockets? **CORE**

The traffic is entirely one-directional — the server announces that new data exists, the browser never pushes back. SSE is plain HTTP, so it passes through proxies and load balancers that need special configuration for WebSocket upgrades, and `EventSource` reconnects automatically, so there is no retry logic to write or get wrong. WebSockets would add bidirectional capability that nothing uses.

Measured latency from cycle completion to browser push is **0.42 s**. A 60-second poll remains as a fallback, and the header shows whether the stream is connected.

8.3 A slow client could back up your event stream. How is that handled?

Each connected browser gets its own bounded queue of eight messages. If a client falls behind, `put_nowait` raises and that update is dropped for that client only — it does not block the publisher or grow memory without limit. Since each message is only a signal to refetch, a dropped one is harmless: the next one triggers the same refresh. A 20-second keep-alive comment prevents idle proxies closing the connection.

8.4 How does the frontend share state across pages?

A React context provider mounted once in the root layout owns the selected district, its live data, connection state and theme. The earlier Vite build threaded these through a router component into all nine pages as props; in the App Router there is no shared parent to drill through, and re-fetching per page would be wasteful. The provider persists across navigations, so switching pages does not re-fetch.

8.5 You said the events panel wasn't realtime. What was wrong? **HARD**

The Current Floods page has two halves. The top read from the shared context and updated on the SSE signal. The **Correlated Flood Events** panel below used its own hook with a 60-second poll and ignored the stream entirely — so one half of a single page could be a minute behind the other.

The fix was to have the context publish a `cycle` counter from the one shared `EventSource`, and have the events hook depend on it. Deliberately *not* a second `EventSource` per component: browsers cap concurrent connections per origin, and one stream already carries the signal.

8.6 How are locations handled now that they are not hardcoded?

Originally a static list of seven districts, seeded from what the IMD mock reports. That was appropriate while IMD was the only source, but OpenWeather works at any coordinate, so the limit had become artificial.

Now `/districts/search` geocodes any place, `/districts/track` persists it, and the collector picks it up on the next cycle with no restart. A tracked location gets live weather but **not** IMD warnings, so the response sets `imd_covered: false`, the IMD slots are nulled rather than left empty, and the UI marks it "Live weather only" — otherwise a blank panel reads as data that has not arrived yet.

8.7 "Use my location" once reported the wrong place. What happened? **HARD**

It compared the GPS fix against the seven followed districts and picked the nearest. From Telangana, Wayanad in Kerala genuinely *is* the closest of those seven — so the answer was the closest known point, and wrong.

It now reverse-geocodes the actual coordinates through OpenWeather, follows that place if it is new, and switches to it. The nearest-neighbour code was deleted rather than patched, because the approach itself was the bug.

9 · Critical & Uncomfortable Questions

These decide the grade. Each is answerable — but only by conceding the real limitation first and then showing it was a considered decision.

9.1 IMD is a mock server. Is any of this real? **HARD**

Part of it is genuinely live and part is simulated, and the interface says which is which.

Real: OpenWeather observed conditions and five-day forecasts at real coordinates for every followed location; the 2,677 chunks are from genuine NDMA, IMD and NDRF publications; the model is trained on 4,548 real gauged flood events; Gemini answers are real.

Simulated: IMD warnings, nowcasts and district rainfall, because the real API requires government credentials that are still pending.

The mock replicates the real endpoint contracts documented in `imd_api_reference.md`, so switching over is a change of `IMD_BASE_URL` plus an auth header. Crucially, the UI badges live figures `LIVE` and marks IMD-uncovered locations, so a mocked number is never presented with the same authority as a measured one.

9.2 What is the weakest part of this project? **HARD**

The ML model. ROC-AUC 0.575 on unseen catchments is a weak signal, and the reason is structural rather than a modelling mistake: rainfall alone is an upstream proxy for flooding. The causal variables — river level, discharge, soil moisture, reservoir storage — come from CWC, which is not yet integrated. Until that data exists, no amount of model tuning closes the gap.

The mitigation is architectural: the model contributes 25% of the risk score, not 100%, and its cross-validated score travels with every prediction the API returns.

9.3 What would break first if this went into real use? **HARD**

- **SQLite write contention** once collection frequency or district count rises materially.
- **In-memory SSE subscribers.** Multiple API replicas would each hold their own subscriber set, so a client connected to replica A would miss a cycle completed on replica B. Fixing that needs Redis pub/sub.
- **OpenWeather quota.** Two calls per location per cycle; at 60-second intervals the free 60-per-minute limit is reached at about 30 locations.
- **No authentication.** Every endpoint is open, including the ones that write.
- **District-name matching** would fragment as coverage grows and spellings diverge.

9.4 What are the security concerns? **HARD**

The honest answer is that there is no auth layer, and the fake Logout button was removed rather than left implying one exists. Keys live in a gitignored `.env` and never reach the client bundle, since the browser only calls same-origin `/api/*`. Input is validated by Pydantic and all database access goes through the SQLAlchemy ORM, so injection is not a live concern.

What is missing before real deployment: authentication and authorisation, rate limiting on the write endpoints, and CORS narrowed from its current development configuration.

9.5 How do you know any of this works? What testing exists? HARD

Verification has been by direct exercise of the running system rather than an automated suite, and that is the main engineering gap. What was verified:

- Trigger rules exercised across every severity band with synthetic observations.
- The full realtime chain proved end to end — a district switched to Red in the source was detected by the scheduler unattended, opened a `severe/active` event at 0.98 confidence from three independent pieces of evidence, was scored, summarised and pushed to the stream in 0.42 s.
- Every error path of the chat client unit-checked: network down, 422, 503, opaque 500, object-shaped detail.
- Model evaluation via grouped cross-validation rather than a single split.

What is missing: pytest coverage of the correlation engine, a retrieval-quality harness, and frontend component tests. The correlation engine is the priority — it holds the most conditional logic and the most subtle bugs.

9.6 Tell me about a bug you found and fixed. CORE

Several, and they are worth having ready because they demonstrate the system was actually exercised rather than merely assembled.

- **Evidence double-counting.** One basin forecast split across four sub-basins inflated confidence from 0.25 to 0.85. Fixed with per-type diminishing returns; the event correctly settled at 0.53.
- **A count that could not see its own writes.** The diminishing-returns fix silently did nothing at first — the session runs with `autoflush=False`, so the query counting existing evidence could not see rows added earlier in the same batch. Every contribution was being treated as the first of its type.
- **A permanently paused scheduler.** `next_run_time=None` in `APScheduler` means *paused*, so the timer would never have fired in production.
- **Prediction rows all showing one date.** IMD's QPF response is a list of sub-basins, not days; the code read one day per sub-basin and stamped them all with the same issue date.
- **A timezone off-by-one.** `toISOString()` converts local midnight to UTC, shifting every forecast date back a day.

9.7 If you had another month, what would you build?

1. **CWC integration** — river levels and discharge. It improves the model, replaces the estimated river-level figures currently labelled as proxies, and is the single highest-value gap.
2. **A test suite**, starting with the correlation engine.
3. **A retrieval evaluation harness** so RAG changes are justified by metrics rather than by anecdote.
4. **Authentication** and per-user saved locations with notifications.
5. **Map visualisation** — Leaflet with events, stations and basin layers, which the design document specifies and which is a real gap for a geographic product.
6. **Postgres with pgvector**, consolidating relational and vector storage.

9.8 What did you learn?

The most valuable lesson was that **verifying data quality precedes modelling**. It would have been easy to train on the 10,000-row dataset, report accuracy near 50% as though it were a starting point, and never look at the feature importances. Recognising that uniform importance meant random labels — and then finding a genuinely usable dataset instead — mattered more than any tuning that followed.

The second was that **honest interfaces are a design requirement, not decoration**. Marking live data `LIVE`, labelling interpolated features, showing why an event exists, and reporting a weak AUC alongside the number it produces all make the system trustworthy in a way that a higher headline figure would not.